

Application development for Android

Jetpack Compose exercise

Dependency injection

Photos: gallery and camera

Image processing libraries

Geolocation

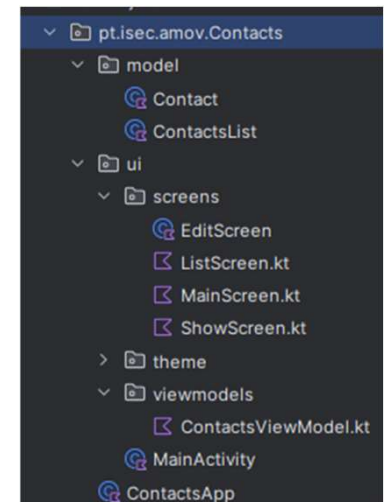
Contacts application

- Create an application to manage contacts
 - A contact will be defined by
 - 1st version: name, e-mail, phone and birthday (can be `null`)
 - 2nd version: additionally, add an image of the contact
 - 3rd version: include geolocation information about the last times a contact was met
 - The model data must be platform independent

Project organization

(Update Android Studio to the latest version!!)

- Create an *Android* project choosing "*Empty Activity*" for the first *Activity*
- Create a project packaging organization that allows to separate application concerns (1st version)
 - `<base package>`
 - `model`
 - The classes in this package cannot import platform-specific libraries
 - `ui`
 - `screens`
 - include all the composables which represent application screens
 - `theme`
 - already created with the project
 - include configurations on generic appearance characteristics
 - `viewmodels`
 - include all *ViewModel* classes
 - ***MainActivity*** class
 - already created with the project
 - ***ContactsApp*** class (extends *Application*)
 - Create this class and register it in the manifest file



Model

- Create a serializable class to represent a contact

```
class Contact(  
    var name: String,  
    var email: String,  
    var phone: String,  
    var birthday: Date?=null,  
) : Serializable
```

Model

- Create a serializable class to represent the list of contacts

```
class ContactsList : Serializable {  
    private val contacts = mutableListOf<Contact>()  
  
    fun addContact(contact: Contact) {  
        contacts.add(contact)  
    }  
  
    fun getContacts(): List<Contact> {  
        return contacts.toList()  
    }  
}
```

Model

- Make it possible to sort the list by contact's name, taking into consideration the special characters of the current device language by using:

```
fun getContacts(): List<Contact> {  
    val collator = Collator.getInstance(Locale.getDefault()).apply {  
        strength = Collator.PRIMARY  
    }  
  
    return contacts.sortedWith { contact1, contact2 ->  
        collator.compare(contact1.name, contact2.name)  
    }  
}
```

Model

- Add methods to save and restore the data model

```
class ContactsList : Serializable {
    /* ... */
    fun save(destination: OutputStream) {
        try {
            ObjectOutputStream(destination).use { oos ->
                oos.writeObject(this)
            }
        } catch (_: Exception) { }
    }
    companion object {
        fun load(source : InputStream) : ContactsList? {
            try {
                ObjectInputStream(source).use { ois ->
                    return ois.readObject() as ContactsList
                }
            } catch (_: Exception){ return null }
        }
    }
}
```

Application data

- Create an instance of the model data in the application object
 - To avoid slow application startup, the instance of the model should be created only when it is first accessed
- Provide a method to save data

```
class ContactsApp : Application() {
    companion object {
        private const val DATAFILE = "contacts.bin"
    }
    private val _contactsList by lazy {
        try {
            openFileInput(DATAFILE)?.let { srcFile -> ContactsList.load(srcFile) }
        } catch (_: Exception) {
            null
        } ?: ContactsList()
    }
    val contactsList get() = _contactsList

    fun saveData() {
        try {
            contactsList.save(openFileOutput(DATAFILE, MODE_PRIVATE))
        } catch (_: Exception) {}
    }
}
```


View Model

- Create a `ViewModel` class that receives a data model instance via dependency injection

```
class ContactsViewModelFactory(  
    private val contactsList : ContactsList  
) : ViewModelProvider.Factory {  
  
    @Suppress("UNCHECKED_CAST")  
    override fun <T:ViewModel> create(modelClass: Class<T>): T {  
        return ContactsViewModel(contactsList) as T  
    }  
}  
  
class ContactsViewModel(  
    val contactsList : ContactsList  
) : ViewModel() {  
  
}
```

View Model (Activity)

- Create an instance of the `ViewModel` for use within the `MainActivity` context
 - In the context of this application, only one instance of the `ViewModel` will be created, and it will be shared across multiple screens (*composables*)

```
class MainActivity : ComponentActivity() {
    private val app by lazy { application as ContactsApp }
    private val viewModel : ContactsViewModel
        by viewModels { ContactsViewModelFactory(app.contactsList) }
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AMovContactsTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Text("Hello World", modifier = Modifier.padding(innerPadding))
                }
            }
        }
    }
    override fun onPause() {
        super.onPause()
        app.saveData()
    }
}
```

UI Interface

- The application should display a main screen with a list of contacts
 - This list should present each contact as a `Card`
 - By default, only the contact's name will be shown
 - Include an `IconButton` in the `TopBar` to switch between this view and an expanded view to show more of the contact's information
- When a `Card` is clicked, a new screen should display all the information related to the selected contact
 - On this screen, an *Edit* `IconButton` in the `TopBar` should allow the user to edit the contact's information

UI Interface (cont.)

- In the main screen, an *Add* `IconButton` should enable the user to create a new contact, using a new screen for this operation
 - Use the same screen for creating a new contact or editing an existing one
 - On this screen, a *Done* `IconButton` should confirm the changes, update the global contact list, and return the user to the main screen
- All screens should display a back navigation icon (back arrow) in the `TopBar`, except the main screen

Main Screen

- Create a *Composable* to represent the main screen: `MainScreen`
- This screen will act as the base for the multiple screens in the application, including:
 - The `Scaffold` composable
 - Management of the *Top Bar* icons
 - Navigation management between screens (*List*, *Show* and *Edit*)
- This composable receives by parameter the reference for the `View Model`

```
@Composable
fun MainScreen(
    viewModel: ContactsViewModelFactory,
    modifier: Modifier = Modifier
) {
    Scaffold(modifier = modifier.fillMaxSize()) {
        innerPadding ->
            Text( "Hello World", modifier = Modifier.padding(innerPadding))
    }
}
```

Navigation

- When managing the display of different screens within an *Activity*, two main components become essential: the `NavController` and the `NavHost`
- To access these and related navigation components, include the following dependency in the application module `build.gradle` file:

```
implementation("androidx.navigation:navigation-compose:2.8.3")
```

Navigation

- **The `NavHostController` (a subclass of `NavController`) is the core element for managing *Navigation* within an application**
 - Manages and remembers the current screen
 - Maintains the back stack of composable screens
 - Preserves the state of each screen in the navigation stack
- **To create a `NavHostController` instance, use:**

```
val navController = rememberNavController()
```

Navigation

- A `NavHost` links a `NavController` with a navigation graph, which defines the composable destinations and their routes
- Each destination in the navigation graph is associated with a unique route
- `NavController` provides several methods, e.g.:
 - `navigate("route")`
 - `navigateUp()`
 - `popBackStack()`
 - `popBackStack("route", inclusive : Boolean)`
 - `currentBackStackEntryAsState()`
 - `addOnDestinationChangeListener(listener), ...`
- When navigation occurs between composables, the `NavHost` automatically recomposes its content to reflect the new destination

```
NavHost(navController, startDestination = "list",...) {  
    composable("list") { ListScreen(...) }  
    composable("show") { ShowScreen(...) }  
    ...  
}
```


Main Screen

- Include the NavHost in the MainScreen
 - Apply the Scaffold's innerPadding value to the NavHost composable

```
@Composable
fun MainScreen(
    viewModel: ContactsViewModel,
    navController: NavHostController = rememberNavController(),
    modifier: Modifier = Modifier
) {
    Scaffold(modifier = modifier.fillMaxSize()) { innerPadding ->
        NavHost(
            navController = navController,
            startDestination = "list",
            modifier = Modifier.padding(innerPadding)
        ) {
            composable("list") { Text("List screen") }
            composable("show") { Text("Show screen") }
            composable("edit") { Text("Edit screen") }
        }
    }
}
```

Main Screen

- Add and configure the *Top Bar* to include the application title and the buttons for navigation and contextual actions related to each screen
 - Change the `startDestination` of the `NavHost` to test each screen *Top Bar* buttons, that will be included in the next steps

```
Scaffold(  
    topBar = {  
        TopAppBar(  
            title = { Text("Contacts") },  
            colors = TopAppBarDefaults.topAppBarColors(  
                containerColor = Color(0,224,255),  
                titleContentColor = Color(0,0,128),  
            ),  
            navigationIcon = { },  
            actions = { },  
        )  
    },  
    ...  
)
```

Navigation Icon

- Include an *Arrow Back* `IconButton` to return to the previous screen
 - This icon cannot appear in the *Main Screen*

- Create a state variable to be aware of changes to the current screen:

```
val currentScreen by navController.currentBackStackEntryAsState()
```

- In the `TopAppBar` object:

```
navigationIcon = {  
    if (currentScreen?.destination?.route != "list") {  
        IconButton(  
            onClick = { navController.navigateUp() }  
        ) {  
            Icon(  
                Icons.AutoMirrored.Filled.ArrowBack,  
                contentDescription = "Back"  
            )  
        }  
    }  
},
```

Action buttons

- Define the already mentioned action buttons to the *Top Bar*
 - *List Screen*: "expand/collapse" and "add"
 - *Show Screen*: "edit"
 - *Edit Screen*: "done/save"

```
actions = {  
  when (currentScreen?.destination?.route) {  
    "list"->{ ... }  
    "show"->{ ... }  
    "edit"->{ ... }  
  }  
}
```

Action buttons – List Screen

- Button to expand/collapse cards

```
IconButton(onClick = { }) {  
    Icon(  
        Icons.AutoMirrored.Filled.List,  
        contentDescription = "Expand/Collapse"  
    )  
}
```

- Create a state variable to manage the expand mode and change its value with the previous button

```
var showExpanded by remember { mutableStateOf(false) }
```

Action buttons – List Screen

- Button to add a new contact

```
IconButton(  
    onClick = { navController.navigate("edit") }  
) {  
    Icon(  
        Icons.Filled.Add,  
        contentDescription = "Add Contact"  
    )  
}
```

Action buttons – Show Screen

- Button to edit the selected contact

```
IconButton(  
    onClick = { navController.navigate("edit") }  
) {  
    Icon(  
        Icons.Filled.Edit,  
        contentDescription = "Edit Contact"  
    )  
}
```

Action buttons – Edit Screen

- Button to confirm the changes and return to the *List Screen*

```
IconButton(  
    onClick = {  
        navController.popBackStack("list", inclusive=false)  
    }  
) {  
    Icon(  
        Icons.Filled.Done,  
        contentDescription = "Save Contact"  
    )  
}
```

- The `popBackStack("list")` command will remove from the back stack all the screens stored on the path from the *List Screen* up to the current screen
 - The `inclusive` parameter set to `false` is used to stay on the *List Screen* without returning to the previous screen of *List Screen*

List Screen

- Create the `ListScreen` composable to list all the contacts through cards
- The composable function should receive as parameters:
 - the list of contacts
 - the current expand mode (if the cards are to be shown expanded or collapsed)
 - a *lambda expression* with the code to execute when a contact card is clicked

```
@Composable
fun ListScreen(
    contacts : List<Contact>,
    showExpanded : Boolean = false,
    onSelectContact : (Contact) -> Unit = {},
    modifier: Modifier = Modifier
) {
}
```

List Screen

- Use a `LazyColumn` to display the contacts
 - Each contact card will be managed by an independent composable

```
LazyColumn(  
    modifier = modifier.fillMaxSize(),  
) {  
    items(  
        contacts,  
        key = { contact -> contact.hashCode() }  
    ) { contact ->  
        ContactCard(  
            contact = contact,  
            showExpanded = showExpanded,  
            onSelectContact = onSelectContact,  
        )  
    }  
}
```

List Screen – Card

- The `Card` composable is a basic card component that provides a simple container with optional elevation

```
@Composable
fun ContactCard(
    contact: Contact, showExpanded: Boolean,
    onSelectContact: (Contact) -> Unit,
    modifier: Modifier = Modifier
) {
    val formatter = SimpleDateFormat("yyyy.MM.dd", Locale.getDefault())
    Card(
        modifier = modifier.fillMaxWidth().padding(8.dp), elevation = CardDefaults.cardElevation(4.dp),
        colors = CardDefaults.cardColors(
            containerColor = Color(255, 224, 192)),
        onClick = { onSelectContact(contact) }
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            verticalArrangement = Arrangement.SpaceBetween
        ) {
            Text(text = contact.name, fontSize = 20.sp)
            if (showExpanded) {
                Row(
                    modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween
                ) {
                    Text(text = contact.email, fontSize = 14.sp)
                    Text(text = contact.phone, fontSize = 14.sp)
                }
                contact.birthday?.let { birthday ->
                    Text(
                        modifier = Modifier.fillMaxWidth(), textAlign = TextAlign.Center,
                        text = formatter.format(birthday), fontSize = 14.sp
                    )
                } ?: Text(modifier = Modifier.fillMaxWidth(),
                    textAlign = TextAlign.Center, text = "----.--.--",fontSize = 14.sp)
            }
        }
    }
}
```

List Screen – Card

```
if (showExpanded) {  
    Row(  
        modifier = Modifier.fillMaxWidth(),  
        horizontalArrangement = Arrangement.SpaceBetween  
    ) {  
        Text(text = contact.email, fontSize = 14.sp)  
        Text(text = contact.phone, fontSize = 14.sp)  
    }  
    contact.birthday?.let { birthday ->  
        Text(  
            modifier = Modifier.fillMaxWidth(),  
            textAlign = TextAlign.Center,  
            text = formatter.format(birthday),  
            fontSize = 14.sp  
        )  
    } ?: Text(modifier = Modifier.fillMaxWidth(),  
        textAlign = TextAlign.Center,  
        text = "----.--.--",  
        fontSize = 14.sp)  
}
```

Main Screen → List Screen

- In the MainScreen's NavHost call the ListScreen composable function with the correct parameters

```
NavHost(  
    navController = navController,  
    startDestination = "list",  
    modifier = Modifier.padding(innerPadding)  
) {  
    composable("list") {  
        ListScreen(  
            contacts = viewModel.contactsList.getContacts(),  
            showExpanded = showExpanded,  
            onSelectContact = { contact ->  
                Log.d("MainScreen",  
                    "onSelectContact:${contact.name}")  
            }  
        )  
    }  
    ...  
}
```

Test

- Test the application by creating some fake contacts in the `ContactsApp` object
 - Include a method in the model to allow resetting the contact list, so fake contacts are not saved

View Model – Select contact

- Add a method to the View Model to select a contact
 - The selected contact should be stored internally so that it can be shared by multiple application screens
- Call this method in the context of the `onSelectContact` code defined in the *Main Screen*

- *ContactsViewModel*:

```
var currentContact : Contact? = null
```

```
fun selectContact(contact : Contact) {  
    currentContact = contact  
}
```

- *MainScreen*:

```
composable("list") {  
    ListScreen(  
        contacts = viewModel.contactsList.getContacts(),  
        showExpanded = showExpanded,  
        onSelectContact = { contact ->  
            viewModel.selectContact(contact)  
            navController.navigate("show")  
        }  
    )  
}
```

Show Screen

- Create the `ShowScreen` composable function to display the information related to the selected contact
 - This composable should receive as parameter the contact to display
- Call this composable function in the "show" branch of the `NavHost`

```
@Composable
fun ShowScreen(
    contact: Contact,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxSize().padding(16.dp)
    ) {
    }
```

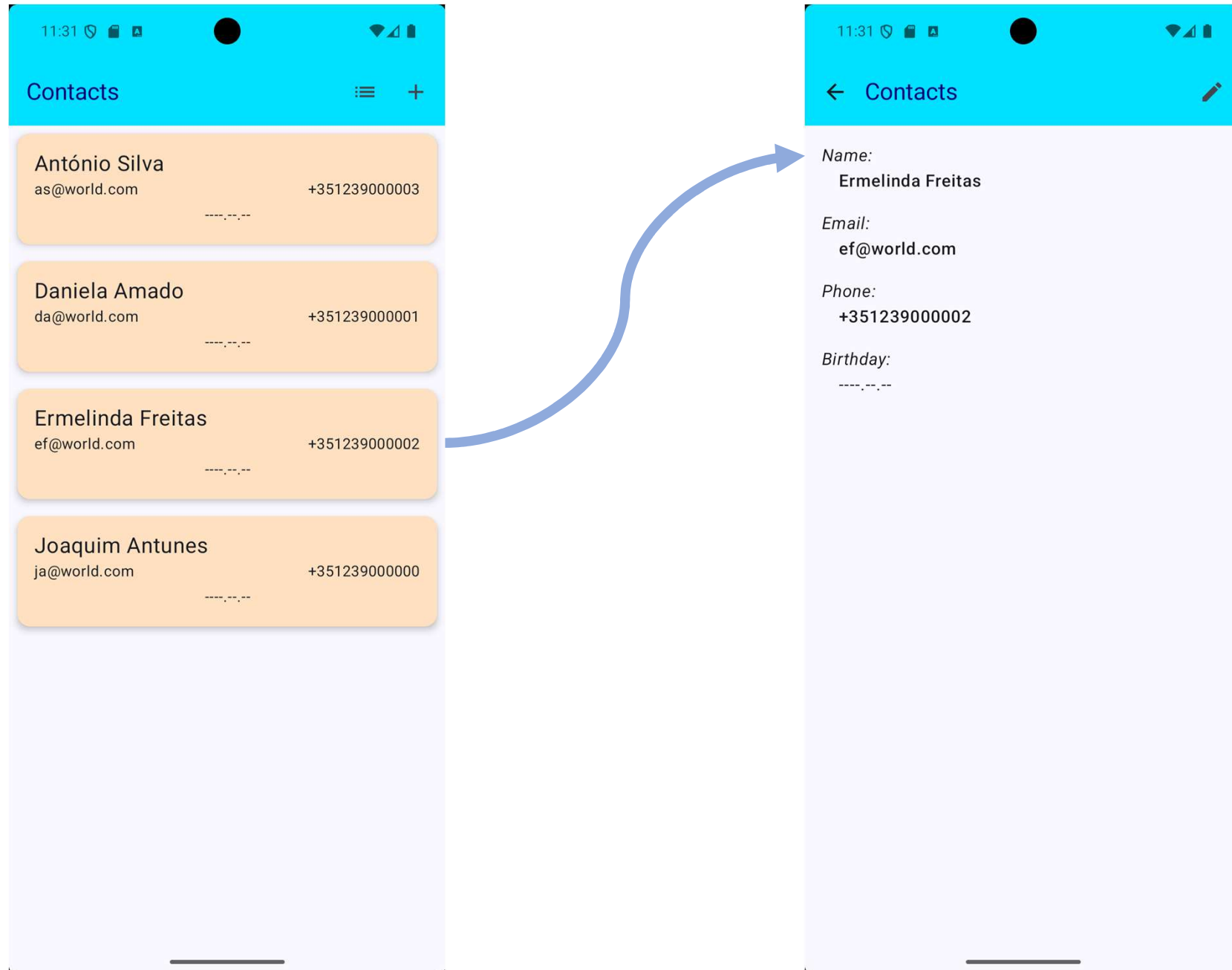
- *MainScreen*:

```
composable("show") {
    viewModel.currentContact?.let { contact ->
        ShowScreen(contact = contact)
    } ?: Text("No contact selected")
}
```


Show screen

- Display all contact information, including:
 - *Name*
 - *E-mail*
 - *Phone*
 - *Birthday*, if defined (otherwise, display "--.--.--")
- Format
 - Field name: *italic*, font size 16.sp
 - Field data: *bold (medium)*, font size 16.sp, indented by 16.dp

Application screens



Edit Screen

- The Edit Screen will be used to enter the information for a new contact but also to edit an existing one
- The user can enter all the contact information but, if the back button is pressed, the information changed will not be saved
 - When creating a new contact that operation will be aborted (no new contact will be created)
- Therefore, instead of using the `currentContact` : `Contact`, that already exists in the View Model, independent temporary variables should be created to store the values
 - The new data is only stored if the edit operation is confirmed
 - For an existing contact the temporary information will override the existent
 - For a new contact, a new `Contact` object must be created with the new information entered by the user

TextField and OutlinedTextField

- To use `TextField` or `OutlinedTextField`, state variables (`String` type) should be created to store the values

```
var name = remember {mutableStateOf("")}
```

```
OutlinedTextField(  
    value = name.value,  
    isError = name.value.isEmpty(),  
    label = {  
        Text("Name:")  
    },  
    onValueChange = { newText ->  
        name.value = newText  
    },  
    modifier = Modifier  
        .fillMaxWidth()  
)
```

DatePicker

- For the `DatePicker` a state variable of the type `DatePickerState` is needed

```
var datePickerState = rememberDatePickerState()
```

```
DatePicker(  
    state = datePickerState,  
)
```